

Part 1

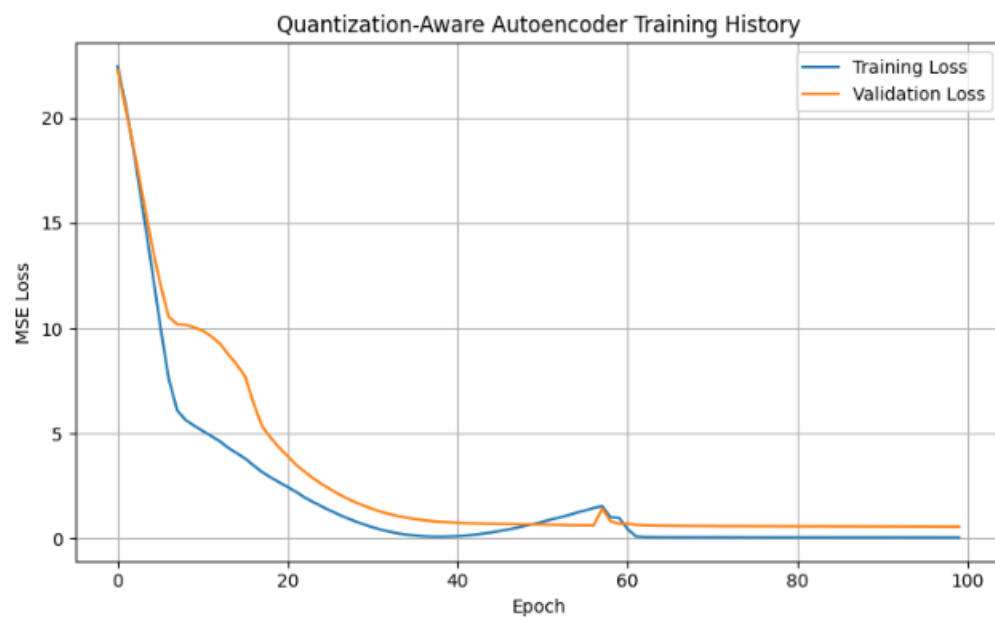
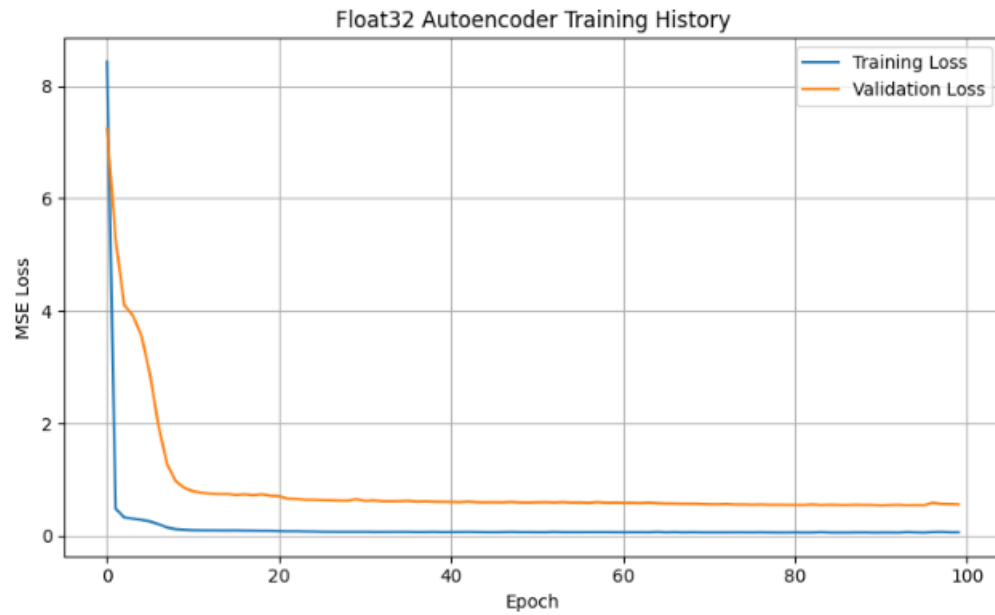
Edge Impulse Project Link: <https://studio.edgeimpulse.com/public/994117/live>

1. **Raw Inputs:** The raw inputs are 3-axis accelerometer data from the Arduino's IMU sensor. These are **accX**, **accY**, and **accZ**, captured at 100 Hz with 2000 ms window size and 200 ms stride.
2. The features passed into the NN classifier are spectral feature and there are a total of 33 input features. At least five feature names used to train the classifier are:
 - accX RMS
 - accX Peak 1 Height
 - accX Spectral Power 2.0-5.0
 - accY Spectral Power 2.0-5.0
 - accZ Spectral Power 2.0-5.0
3. The classes of the NN classifier outputs are **nominal** and **off**, indicating the fan's state as on or off.
4. The anomaly detector uses 3 spectral features named:
 - accX Peak 1 Height
 - accY Spectral Power 2.0-5.0
 - accZ Spectral Power 2.0-5.0
5.

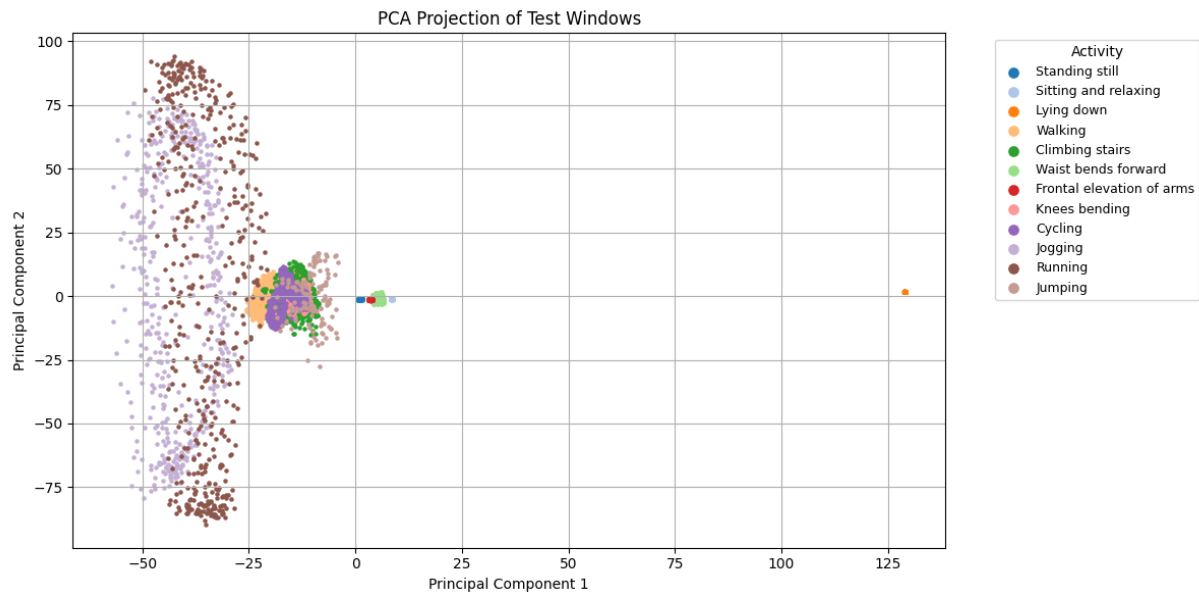
```
Reconstruction error: 0.143346
Result: Normal activity
Reconstruction error: 0.143336
Result: Normal activity
```
6. The classifier detects fan movement but still should not be trusted unconditionally. Confident interpretations can be made when the input data closely resembles the training distribution, such as data that reflects a clear on or off state for the fan. The classifier may be unreliable if the IMU used to collect training data and the IMU to implement the classifier have different operating conditions, such as environment, mechanical wear, or accumulation of dust that affects fan movement.

Part 2

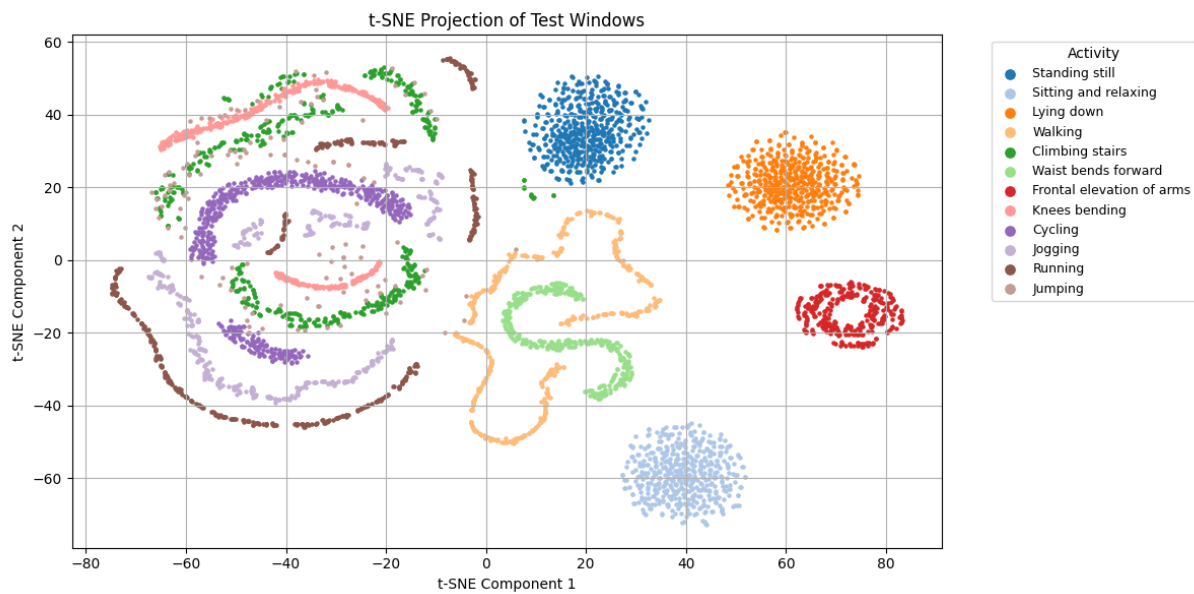
Training and Validation Loss Curves



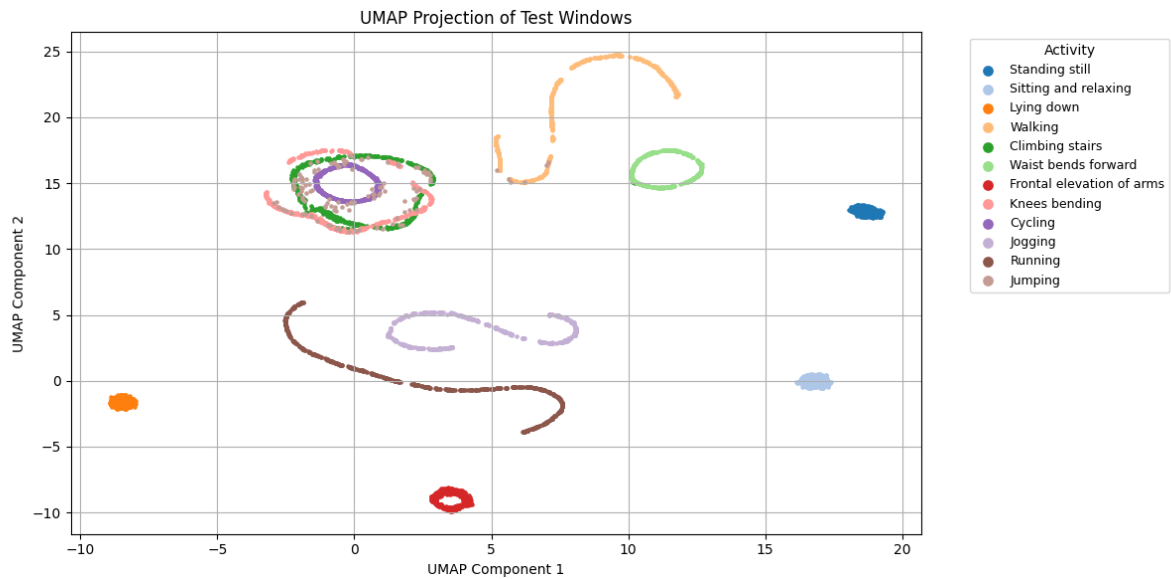
PCA Visualization



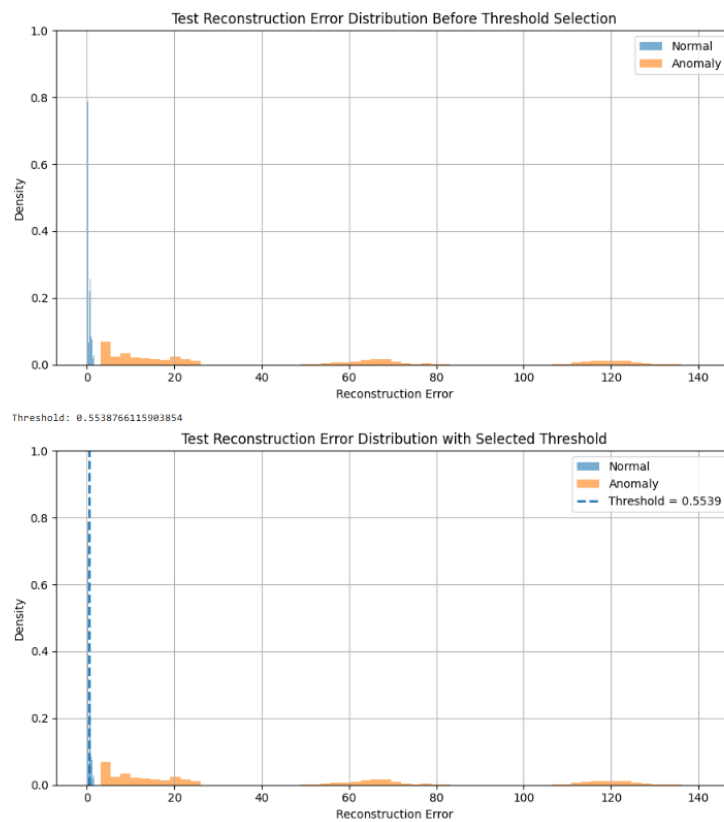
T-SNE Visualization



UMAP Visualization



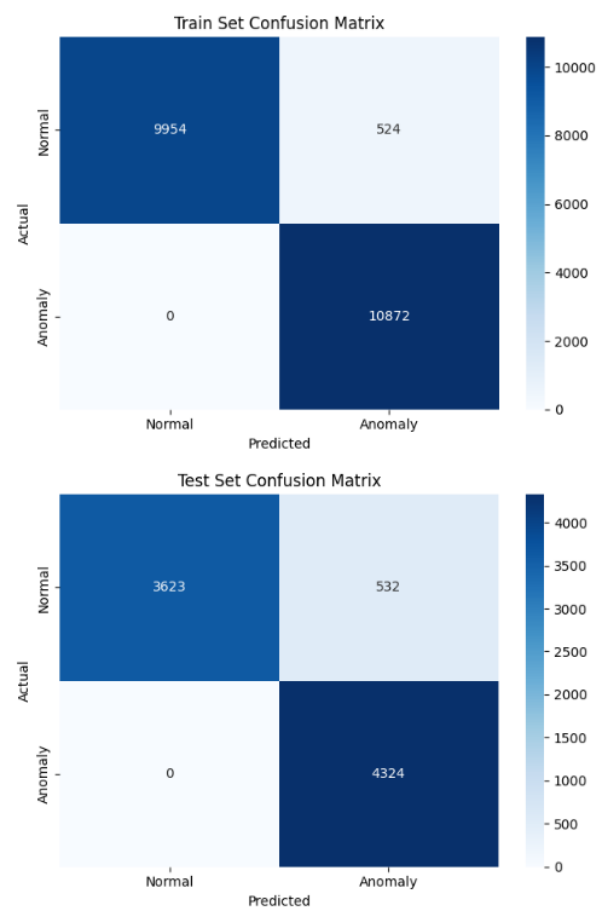
Reconstruction Error Distribution



Selected Anomaly Threshold

0.5538799115903854

Confusion Matrix



Evaluation Metrics

```
Input quantization: (0.08135765045881271, 18)
Output quantization: (0.07862449437379837, 7)
INFO: Created TensorFlow Lite XNNPACK delegate for CPU.
Train set evaluation: int8 model
      precision    recall  f1-score   support

 Normal (0)         1.00      0.95      0.97      10478
 Anomaly (1)         0.95      1.00      0.98      10872

 accuracy          0.98
 macro avg         0.98      0.97      0.98
 weighted avg      0.98      0.98      0.98

Test set evaluation: int8 model
      precision    recall  f1-score   support

 Normal (0)         1.00      0.87      0.93      4155
 Anomaly (1)         0.89      1.00      0.94      4324

 accuracy          0.94
 macro avg         0.95      0.94      0.94
 weighted avg      0.94      0.94      0.94
```

Arduino Serial Monitor Output

Normal Activity

```
Reconstruction error: 0.138588  
Result: Normal activity  
Reconstruction error: 0.138597  
Result: Normal activity  
Reconstruction error: 0.138636  
Result: Normal activity  
Reconstruction error: 0.143346  
Result: Normal activity  
Reconstruction error: 0.143336  
Result: Normal activity
```

Anomaly Activity

```
Reconstruction error: 3.298009  
Result: Anomaly detected  
Reconstruction error: 3.270626  
Result: Anomaly detected  
Reconstruction error: 3.298897  
Result: Anomaly detected  
Reconstruction error: 3.250020  
Result: Anomaly detected
```

Part 3

1. The threshold was set using the 95th percentile of the training normal errors, which was approximately 0.554. This threshold guarantees that 95% of normal activity windows are correctly classified as normal, flagging anything else above that level as an anomaly.
2. Since the autoencoder is trained on normal activity windows exclusively, passing a normal window through the model produces a reconstructed output with a root mean squared error that is low. However, passing an anomalous window through generates a high root mean squared error because the autoencoder is seeing patterns not recognized/seen from training, resulting in a high reconstruction error that reflects the anomalous state.
3. The important information that must be preserved from Python include:
 - The int8 quantized model weights (autoencoder_model.cc)
 - The input dimension: 300 (flattened 100x3)
 - Window size and stride: (2000, 100)
 - Reconstruction error threshold computed from training data: 0.554
 - Sensor channel selection (5,6,7)
 - Input quantization scale
4. It is important for the Arduino sketch to use the same preprocessing assumptions as the notebook because even small mismatches can cause significant reconstruction errors that may make the model's classification unreliable. Consistency in window size, sensor axes, sample order, and quantization mapping between Python and the Sketch ensure the same input for the model at inference time and translates the model's learned representations to hardware the best.
5. One of the most significant limitations of implementing anomaly detection on a microcontroller is the finality of the model. Things like the reconstruction error threshold and weights of the model and the memory of the MCU are fixed until a new model is flashed or the hardware is updated, meaning the model cannot independently adapt to sensor characteristics over time.